Technical Report: Non-Invasive Security Review of the SpaceEdu Demo Application

Author: Dev Shishodia
Programme: RedKross Research Foundation Internship, Task 1
Intern ID: 2426
Report date: 25 October 2026

1. Objective

This task required a non-invasive security review of a supplied demo site, or of a site with written permission, or of a local demo. The review had to record observable security headers, HTTPS behaviour, exposed information, and usability and security concerns. It had to avoid scanning, exploitation and login bypass attempts. Deliverables are scope and authorisation notes, observations with evidence, severity and priorities, and actionable recommendations.

No third-party target was supplied, so the author built a local demo application (SpaceEdu, an online space-science course platform) and reviewed it. The application is deliberately unhardened and was built with an AI coding assistant. This report documents what a passive reviewer can learn about such an application and how to fix it.

2. Scope and Authorisation

Item	Detail
Target	SpaceEdu demo application (self-built, AI-assisted)
URLs	https://localhost:8443 (Caddy TLS proxy), http://localhost:8081 (redirect), http://localhost:8080 (direct backend)
Owner and authoriser	Dev Shishodia, who built, owns and operates the target. No third-party system is involved.

Item	Detail
Environment	Local machine only: Go backend, PostgreSQL 16 (Docker), static HTML/JS frontend, Caddy 2 reverse proxy using its internal CA
Baseline	Git tag `baseline-pre-review`

In scope: viewing pages in a browser, reading response headers (DevTools and single `curl` requests) inspecting the TLS certificate and redirect behaviour, inspecting local storage and cookies, reading page source, normal use of signup, login, course listing and checkout (test card only), replaying the author's own unmodified requests in Burp Suite Repeater.

Out of scope: port scanning, directory brute-forcing, fuzzing, exploit attempts, authentication bypass, brute-force or rate-limit testing, injection payloads, load generation, automated scanners (Burp Scanner and Intruder were not used), and any non-local host.

3. Background and Research

The checklist was built from established guidance rather than ad-hoc ideas:

- **OWASP Secure Headers Project** and **HTTP Headers Cheat Sheet** define the expected response headers (HSTS, CSP, X-Content-Type-Options, frame protections, Referrer-Policy, Permissions-Policy).
- **RFC 6797** specifies HSTS and explains why a redirect alone does not stop a first-visit downgrade.
- **OWASP HTML5 Security Cheat Sheet** and **Session Management Cheat Sheet** cover token storage (localStorage versus HttpOnly cookies).
- **OWASP Unvalidated Redirects and Forwards Cheat Sheet** and **CWE-601** cover open redirects.
- **OWASP Authentication Cheat Sheet** and **NIST SP 800-63B** cover password policy and throttling.
- **CWE-538** covers sensitive files exposed through a web server.
- **PCI DSS** explains why card data should not touch an application server.
- **RFC 9116** defines `security.txt`.
- **OWASP Risk Rating Methodology** (likelihood times impact) was used for severity.

- **OWASP Web Security Testing Guide** was used for structure, limited to its passive information-gathering and configuration-review parts.

4. Target Description and Preparation

The target has a Go (`net/http`) backend with JWT authentication (HS256) and bcrypt password hashing. PostgreSQL stores users, courses, orders and enrollments. A static HTML/JS frontend (index, courses, login, checkout, enrolled) is served by the Go process. Caddy provides local HTTPS on 8443 and an HTTP redirect on 8081, and proxies to the backend on 8080.

5. Methodology

1. Wrote the scope note before any observation.
2. Collected HTTP evidence with single requests (`curl -I` and `curl`) to the redirect, the HTTPS site, an API route, the direct backend, a non-existent path, and one file path identified during code reading. Output is saved in `evidence/01` to `06`.
3. Inspected the TLS certificate and negotiated connection in Firefox.
4. After a normal login and checkout, inspected browser storage and the network traffic of the author's own requests (Firefox DevTools and Burp Repeater replay).
5. Read the application source to explain or confirm what the observations suggested. Findings that rely on this are labelled **Source-assisted**. Findings seen directly in traffic or the browser are labelled **Observed**.
6. Rated each finding with likelihood times impact, assuming the application were deployed publicly, and noted where the demo context lowers the practical risk.

Severity scale: Critical (trivial unauthenticated compromise path), High (serious weakness with realistic exploitation), Medium (meaningful weakness, needs conditions), Low (hardening gap or minor leak), Info (no direct risk).

Tools: Firefox (DevTools, certificate viewer), curl, Burp Suite Community Repeater, Docker, Go toolchain, Caddy.

6. Findings Summary

ID	Finding	Severity	Basis
F-01	Project root is web-served; `/backend/.env`, source and compose file are reachable	Critical	Observed (single request) + source
F-02	No security headers on any response	Medium	Observed
F-03	Session token and profile stored in localStorage, no cookies	Medium	Observed
F-04	Card number and CVV sent to the app backend; order completed with no payment processor	High (real-world)	Observed
F-05	Open redirect through `login.html?redirect=`	Medium	Source-assisted
F-06	Weak authentication controls (6-character minimum, no throttling, email enumeration)	Medium	Source-assisted
F-07	JWT handling: HS256, 24 h lifetime, no revocation, hardcoded fallback secret	Medium	Observed (token) + source
F-08	Plain-HTTP backend on :8080 reachable directly, bypassing the proxy	Low	Observed
F-09	Misleading "256-bit SSL" trust claim on checkout	Low	Observed
F-10	Silent fallback to hardcoded course data can show or enrol the wrong course	Medium	Observed + source
F-11	PostgreSQL port published to the host with default credentials	Low	Source-assisted

ID	Finding	Severity	Basis
F-12	Informational: Server banner on redirect, default 404, third-party assets without SRI, partial objects in API responses, no security.txt	Info	Observed + source

7. Detailed Findings

F-01 Exposed project files (Critical)

Evidence: `evidence/06-env-exposure-headers.txt` shows HTTP 200 for `/backend/.env` with a non-zero Content-Length. Only the status and headers were recorded; file contents are deliberately not reproduced in this report.
Observation: the static file server is rooted at the project directory instead of a dedicated public folder. Source code, the Docker compose file and the environment file are therefore downloadable without authentication (CWE-538).
Impact: the environment file holds the JWT signing secret and database credentials. Possession of the signing secret would allow forged tokens for any user. This was not attempted because it is out of scope.
Recommendation: serve only a dedicated `public/` directory, keep configuration outside it, rotate the JWT secret and database password, and verify with `curl -I` that the path returns 404.

F-02 Missing security headers (Medium)

Evidence: `evidence/02-https-headers.txt`, `03-api-headers.txt`, `04-direct-backend-headers.txt`.
Observation: no `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` or `frame-ancestors`, `Referrer-Policy` or `Permissions-Policy` on the HTTPS site, API or direct backend. The HTTP-to-HTTPS redirect works (`01-http-redirect.txt`), but without HSTS a first visit can still be downgraded.
Impact: clickjacking, MIME sniffing and a weaker defence against script injection. This matters more because the token is readable by scripts (F-03).
Recommendation: see the header block in section 8.

F-03 Token and profile in localStorage (Medium)

Evidence: `cookie.png`, `token.png`, `user_details.png` (Firefox Storage panel, redacted).
Observation: after login, the JWT and the user object (id, name, email, creation time) are stored in localStorage. The application sets no cookies, so none of the HttpOnly, Secure or SameSite protections apply. Cookies visible in the Burp request (`language`, `cookieconsent_status`, `welcomebanner_status`) were leftovers from another local application, not set by SpaceEdu.
Impact: any script running on the origin can read the token. Combined with F-02, one cross-site scripting bug would expose sessions.
Recommendation: issue the session as a cookie with `HttpOnly; Secure; SameSite=Lax` and add CSRF protection if cookies are used for state-changing requests.

F-04 Card data sent to the application server (High real-world, Medium as a demo)

Evidence: `course_payment.png`.
Observation: `POST /api/checkout` carries `card_number`, `expiry` and `cvv` in the JSON body, and the response returns 201 with order status `completed`. No payment processor is involved, and the application does not validate the card server-side. A well-known test card number was used for all testing.
Impact: a real deployment would put the application in PCI DSS scope and would grant products without payment.
Recommendation: use a hosted payment form (for example Stripe Elements or Checkout) so card data never reaches the server, validate server-side, and mark orders paid only after provider confirmation.

F-05 Open redirect (Medium, source-assisted)

Evidence: the `Referer` in `login_request.png` shows `login.html?redirect=%2Fenrolled.html`, confirming the parameter exists. The source assigns its value to `window.location.href` without validation (CWE-601). It was not exploited.
Impact: phishing through a trusted domain, and possible script execution through a `javascript:` URL.
Recommendation: accept only same-origin relative paths: the value must start with a single `/`, must not start with `//` or `/\`, and a parsed URL must have the page's own origin.

F-06 Weak authentication controls (Medium, source-assisted)

The source enforces a 6-character minimum password, has no rate limiting or lockout, and answers registration of an existing email with a distinct 409 error. None of this was tested dynamically because brute force is out of scope.
Recommendation: at least 8 characters per NIST SP 800-63B with breached-password screening (12 or more preferred), throttle by IP and account, and return a uniform registration response.

F-07 JWT handling (Medium)

Evidence: `login_request.png`. The token header decodes to `{"alg":"HS256","typ":"JWT"}` and the payload contains only `sub`, `iat` and `exp`. The source sets a 24-hour lifetime, has no server-side revocation or logout, and falls back to a hardcoded secret if the environment variable is missing.
Recommendation: fail at start-up if the secret is missing, use a long random secret, shorten the access-token lifetime with refresh and rotation, and add server-side revocation.

F-08 Direct plain-HTTP backend (Low)

Evidence: `04-direct-backend-headers.txt`. The backend listens on all interfaces, so `http://localhost:8080` serves the site in plain text and skips the proxy.
Recommendation: bind the backend to `127.0.0.1` so only the proxy can reach it.

F-09 Misleading trust claim (Low)

Evidence: `course.png` shows "256-bit SSL Encrypted Transaction • Satisfaction Guaranteed". `certificate.png` shows the negotiated suite was TLS 1.3 with AES-128-GCM. No real payment processing exists.
Recommendation: remove or correct claims that the application cannot back up.

F-10 Silent fallback to wrong course (Medium)

Evidence: in `course_payment.png` the checkout page URL carried a course UUID, but the request body contained `"course_id":"earth-101"`. During testing, clicking different courses led to the Earth 101 checkout, and a direct request for a course UUID returned `course not found` even though the course list endpoint returned all six courses. The root cause of the failed lookup was not confirmed; the running process may not have been using the patched query, and this is recorded as a limitation. The frontend source

substitutes hardcoded course data whenever the lookup fails.
Impact: a user can be shown, and charged for, a course they did not choose. A backend outage can also produce a fake success page.
Recommendation: remove the hardcoded fallback and show an error when a course cannot be loaded. Never submit an identifier that was not loaded from the API.

F-11 Database exposure (Low, source-assisted)

The compose file publishes PostgreSQL on host port 5433 with default credentials. Bind it to `127.0.0.1:5433:5432` and use a generated password from a secrets file.

F-12 Informational

The HTTP redirect response includes `Server: Caddy` (`01`). A non-existent path returns Go's default plain-text 404 (`05`). Fonts and images load from Google and CloudFront without Subresource Integrity or a CSP. The checkout response returns a partly empty course object (empty description, `0001-01-01` creation date). The site has no `security.txt` (RFC 9116). Recommendations: remove the Server banner, add a custom error page, self-host or add SRI to third-party assets, return complete objects, publish `security.txt`.

8. Prioritised Remediation Plan

Order	Action	Fixes	Effort
1	Serve only `public/`; rotate JWT secret and DB password	F-01, F-07	Low
2	Add security headers at the proxy	F-02, F-12	Low
3	Remove hardcoded fallback; fix lookup failure	F-10	Low
4	Validate the `redirect` parameter	F-05	Low
5	Bind backend and database to 127.0.0.1	F-08, F-11	Low
6	Move session to an HttpOnly cookie; shorten token lifetime	F-03, F-07	Medium
7	Strengthen password policy, add throttling	F-06	Low to Medium

Order	Action	Fixes	Effort
8	Replace card handling with a hosted payment form; correct trust claims	F-04, F-09	High

Suggested proxy header block (Caddyfile):

```
header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        Content-Security-Policy "default-src 'self'; script-src 'self';
style-src 'self' https://fonts.googleapis.com; font-src
https://fonts.gstatic.com; img-src 'self' data: https:; connect-src 'self';
frame-ancestors 'none'; base-uri 'self'; form-action 'self'"
        X-Content-Type-Options "nosniff"
        Referrer-Policy "strict-origin-when-cross-origin"
        Permissions-Policy "camera=(), microphone=(), geolocation=()"
        -Server
}
```

The pages currently use inline scripts and styles. A strict CSP requires moving them to external files or using nonces; allowing `'unsafe-inline'` as a stopgap weakens the policy.

9. Testing and Evaluation

The review phase was completed as described in section 5. **Remediation was not applied or re-tested at the time of writing**, so no before-and-after results are claimed. The planned verification after fixes is: `curl -I` shows the new headers; `/backend/.env` returns 404; the `redirect` parameter rejects external URLs; and checkout refuses to proceed when a course cannot be loaded.

Positive observations: the HTTP-to-HTTPS redirect works; TLS 1.3 with an ECDSA P-256 certificate (issued by Caddy's local CA, valid for about 12 hours); bcrypt password hashing; parameterised SQL; HTML escaping of rendered data; generic login error messages; and a CORS origin allowlist (source-assisted).

10. Limitations

- The target is self-built, so results demonstrate method rather than a real engagement.

- This was a passive review, not a penetration test. Absence of a finding is not proof of security.
- Authentication throttling, injection, access control between users and session handling under attack were deliberately not tested.
- F-05, F-06, F-07 (parts), F-10 (cause) and F-11 rely on source reading, not independent traffic evidence.
- The root cause of the failed course lookup was not confirmed.
- The TLS certificate comes from a local CA, so certificate findings do not represent production.
- One browser (Firefox) was used. Severity ratings assume public deployment.

11. Conclusion

Even a small AI-generated web application can have serious gaps that a passive reviewer can see without attacking anything. The most serious problem, a file server exposing the project directory, was visible through a single request. Missing headers, token storage in the browser and a checkout that handles card data all follow common patterns in generated code. Most fixes are low effort and are listed in priority order above. A follow-up re-test after remediation would complete the before-and-after comparison.

12. References

1. OWASP Secure Headers Project. https://owasp.org/www-project-secure-headers/
2. OWASP HTTP Headers Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html
3. RFC 6797, HTTP Strict Transport Security. https://www.rfc-editor.org/rfc/rfc6797
4. OWASP HTML5 Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
5. OWASP Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
6. OWASP Unvalidated Redirects and Forwards Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

7. OWASP Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
8. NIST SP 800-63B, Digital Identity Guidelines. https://pages.nist.gov/800-63-3/sp800-63b.html
9. CWE-538 and CWE-601, MITRE. https://cwe.mitre.org/
10. PCI Security Standards Council, PCI DSS. https://www.pcisecuritystandards.org/
11. RFC 9116, security.txt. https://www.rfc-editor.org/rfc/rfc9116
12. OWASP Risk Rating Methodology. https://owasp.org/www-community/OWASP_Risk_Rating_Methodology
13. OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/
14. MDN Web Docs, Content-Security-Policy. https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
15. Caddy documentation. https://caddyserver.com/docs/

Appendix A. Evidence Index

File	Shows
`01-http-redirect.txt`	HTTP to HTTPS redirect response (F-02, F-12)
`02-https-headers.txt`	Response headers of the HTTPS site (F-02)
`03-api-headers.txt`	Response headers of an API route (F-02)
`04-direct-backend-headers.txt`	Direct backend on :8080 (F-08)
`05-404-page.txt`	Default not-found response (F-12)
`06-env-exposure-headers.txt`	Status and headers for the exposed file path (F-01)
`certificate.png`, `cert-1.png`, `cert-2.png`	TLS connection and certificate details
`cookie.png`, `token.png`, `user_details.png`	localStorage contents, redacted (F-03)
`login_request.png`	Login request and response, redacted (F-05, F-07)

File	Shows
`course_payment.png`	Checkout request and response (F-04, F-10)
`course.png`	Checkout page UI (F-09)

Appendix B. Reproduction Commands

```
curl -sI  http://localhost:8081/index.html
curl -skI https://localhost:8443/
curl -skI https://localhost:8443/api/courses
curl -sI  http://localhost:8080/index.html
curl -sk  https://localhost:8443/nonexistent
curl -skI https://localhost:8443/backend/.env
```